

Memo

To

Delft3D users

Date

2026-07-19

Our reference

001

Number of pages

7

Contact person

Johan Reyns

Direct line

E-mail

johan.reyns@deltares.nl

Subject

Slurm MPI Orchestration for MorMerge

1 Purpose and current implementation

This note documents the current Slurm implementation in `mormerge.tcl` for MorMerge calculations with MPI-partitioned DIMR models. The implementation uses one Slurm allocation for the complete MorMerge calculation and starts every condition as a direct DIMR job step. The existing local, SGE, and Torque paths remain separate from this Slurm logic.

All MorMerge conditions must remain active concurrently because they exchange data with the MorMerge server at morphological communication times. The scheduler can reduce the number of physical nodes by placing several condition steps on one node, but it does not serialize the conditions.

The Slurm implementation has these properties:

- `mormerge.tcl` derives both the per-condition MPI geometry and the allocation-wide task density;
- the executable `<dimrexedir>/dimr` is started directly;
- every condition uses an exclusive, exact `srun` job step;
- Slurm selects the physical nodes and CPU sets;
- no condition-specific `--nodelist` is generated;
- several condition steps may share one allocation node when sufficient CPU and memory capacity is requested;
- a failed DIMR or `srun` process is propagated to the final `mormerge.tcl` : Failed status.

2 Execution sequence

When started outside a Slurm allocation with `queueSystem = slurm,mormerge.tcl` generates an `sbatch` script for the complete calculation. That script re-enters `mormerge.tcl` inside the allocation with the internal node count.

Inside the allocation the launch sequence is:

- 1 clone the model input into one directory per condition;
- 2 clear the synchronization directory;
- 3 start the serial MorMerge process for every run id;
- 4 wait for the MorMerge synchronization markers;
- 5 start one direct DIMR job step for every condition;
- 6 wait for all condition and MorMerge processes;
- 7 return a nonzero status when a condition launcher fails.

The MorMerge processes remain ordinary serial child processes of the batch script. DIMR owns the MPI model execution inside the Slurm job step.

3 User-facing configuration

The normal `nodes`, `queueSystem`, `queue`, `timeLimit`, `condition`, `weight`, and `model-directory` settings remain in the `.mm` file. The Slurm MPI configuration is:

nodes Total number of nodes in the Slurm allocation. This is not the number of nodes used by one condition.

dimrexedir Directory containing the `dimr` executable.

dimrargs DIMR geometry metadata and the DIMR configuration file. The `-c` value must equal `slurmTasksPerNode`. The `--NNODES` value must equal `slurmNodesPerCondition`. These two geometry values are validated by `mormerge.tcl`; the configuration filename and supported DIMR options are passed to `dimr`.

slurmControlNodes Node-equivalent capacity excluded when the condition packing factor is derived. This does not isolate or pin a physical controller node. Default: 0.

slurmNodesPerCondition Number of nodes requested by each condition job step. A value of 0 defaults to one node in the optimized scheduler.

slurmConditionsPerNode Maximum number of concurrent condition-step slices accommodated by one allocation node. A value of 0 automatically selects the minimum value needed to fit all conditions in the requested allocation. Default: 0.

slurmTasksPerNode MPI ranks used by one condition on each node assigned to that condition. Default: 1.

slurmCpusPerTask CPUs reserved for each MPI rank. This also sets the OpenMP and common threaded-library limits. Default: 1.

slurmMPI The site-supported value for `srunk --mpi`, for example `pmix` or `pmi2`. A multi-rank direct DIMR condition requires a compatible value. The correct choice depends on the Slurm and MPI builds installed at the site.

slurmModuleCommand Optional site-specific command written into the generated batch script, for example a compiler and MPI module load command. Default: `none`.

4 Resource geometry

The optimized scheduler separates model geometry from allocation packing. The following symbols describe the calculation:

`C` = number of MorMerge conditions
`N` = `slurmNodesPerCondition`

```
P = slurmConditionsPerNode
T = slurmTasksPerNode
H = slurmCpusPerTask
A = nodes - slurmControlNodes
```

Each condition consumes N condition-node slots. All conditions together require:

```
required condition-node slots = C * N
```

When $P = 0$, the scheduler derives:

```
P = ceiling((C * N) / A)
```

An explicit or derived packing factor is valid when:

```
A * P >= C * N
```

The per-condition and per-allocation task counts are:

```
MPI ranks per condition      = N * T
MPI ranks across conditions   = C * N * T
allocation tasks per node     = P * T
allocation CPUs per node      = P * T * H
```

The generated batch script requests:

```
#SBATCH --nodes=<nodes>
#SBATCH --ntasks-per-node=<P * T>
#SBATCH --cpus-per-task=<H>
```

Slurm verifies whether the requested CPU layout can be allocated. MorMerge cannot infer the memory required by a condition, so the selected value of P must also fit the node memory.

5 Direct DIMR job step

For each condition, the generated launcher creates:

```
srun --exclusive --exact --kill-on-bad-exit=1 \
    [--mpi=<slurmMPI>] \
    --nodes=<N> \
    --ntasks=<N * T> \
    --ntasks-per-node=<T> \
    --cpus-per-task=<H> \
    --export=ALL,OMP_NUM_THREADS=<H>,... \
    "<dimrexedir>/dimr" <DIMR options> "<dimr_config.xml>"
```

`--exclusive` prevents concurrent condition steps from using the same CPU set.
`--exact` restricts a step to its requested resources so that other condition steps can use the remaining CPUs on the node. The target Slurm installation must support exact job-step resource allocation for packed runs.

For multi-rank conditions, the launcher first verifies that the configured Slurm MPI plugin exports an MPI rank environment recognized by DIMR. This detects an incompatible Slurm/MPI combination before starting the model. No second MPI launcher is started inside the `srun` step.

6 Supported MPI layouts

6.1 One condition per node

Use one node per condition when a complete model fits on one node and the allocation contains at least one node for every condition:

```
nodes                = 24
slurmNodesPerCondition = 1
slurmConditionsPerNode = 1
slurmTasksPerNode     = 10
slurmCpusPerTask      = 1
dimrargs              = -c 10 --NNODES 1 dimr_config.xml
```

For 24 conditions this runs 10 MPI ranks per condition, requests 10 tasks per node, and runs 240 MPI ranks in total.

6.2 Several conditions per node

Use packing when a node has enough CPU and memory capacity for several complete models. Setting `slurmConditionsPerNode = 0` lets the scheduler derive the minimum packing needed for the requested node count:

```
nodes                = 12
slurmNodesPerCondition = 1
slurmConditionsPerNode = 0
slurmTasksPerNode     = 10
slurmCpusPerTask      = 1
dimrargs              = -c 10 --NNODES 1 dimr_config.xml
```

For 24 conditions the derived packing factor is 2. The allocation requests 20 tasks per node, and Slurm places two exclusive 10-rank condition steps on each node. The total MPI rank count remains 240.

The same calculation on six sufficiently large nodes derives
`slurmConditionsPerNode = 4` and requests 40 tasks per node.

6.3 One condition across several nodes

Use multiple nodes per condition when a partitioned model does not fit on one node or when more distributed MPI capacity is required:

```
nodes                = 8
```

```
slurmNodesPerCondition    = 2
slurmConditionsPerNode    = 1
slurmTasksPerNode        = 16
slurmCpusPerTask          = 1
dimrargs                  = -c 16 --NNODES 2 dimr_config.xml
```

With four conditions, every condition uses 32 MPI ranks across two nodes. The allocation runs 128 MPI ranks in total.

6.4 Packed multi-node conditions

Packing also applies when every condition spans several nodes. In that case each condition consumes one condition slice on every node assigned to its job step:

```
nodes                      = 4
slurmNodesPerCondition    = 2
slurmConditionsPerNode    = 0
slurmTasksPerNode        = 8
slurmCpusPerTask          = 1
dimrargs                  = -c 8 --NNODES 2 dimr_config.xml
```

For four conditions, eight condition-node slots are required. Four allocation nodes therefore produce a packing factor of 2. Each condition uses 16 MPI ranks, the allocation requests 16 tasks per node, and the total rank count is 64.

6.5 Hybrid MPI and OpenMP

Set `slurmCpusPerTask` above 1 when each MPI rank should use OpenMP threads. The number of MPI ranks does not change; only the CPU reservation and thread environment change.

For example, the 24-condition, 12-node packed layout with `slurmCpusPerTask = 2` still launches 20 MPI ranks per node but requests 40 CPUs per node. The generated scripts set `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, and `NUMEXPR_NUM_THREADS` to 2.

7 Intel MPI example

The following example runs 24 conditions for a 10-domain model on 12 nodes. It assumes one MPI rank per model domain and two complete models per node. Module names and the Slurm MPI plugin are site-specific.

```
queueSystem                = slurm
queue                      = normal
timeLimit                  = 12:00:00
nodes                      = 12

dimrexedir                 = /opt/delft3dfm/lnx64/bin
dimrargs                   = -c 10 --NNODES 1 dimr_config.xml
mormergeexedir             = /opt/delft3dfm/lnx64/bin

slurmControlNodes          = 0
slurmNodesPerCondition     = 1
```

```

slurmConditionsPerNode      = 0
slurmTasksPerNode          = 10
slurmCpusPerTask           = 1
slurmMPI                   = pmi2
slurmModuleCommand         = module load <intel-module> <intel-mpi-module>

condition:weight            = condition01 : <weight>
...
condition:weight            = condition24 : <weight>

```

The derived allocation and condition geometry is:

```

conditions                  = 24
condition steps per node    = 2
nodes per condition         = 1
MPI ranks per condition     = 10
allocation tasks per node   = 20
total MPI ranks             = 240

```

`pmi2` is an example, not a universal Intel MPI requirement. Use the plugin and module combination supported by the cluster and compatible with the MPI library used to build DIMR.

8 Completion and failure semantics

MorMerge uses a stream protocol in which the server waits for the next frame. The current server has no normal client-completion frame. After successful condition models close their streams, the server can therefore end with:

```
GetPutArrayError: ABORT Recv ... returns 0 bytes
```

For the current protocol this is the expected server teardown after a condition stream closes. `mormerge.tcl` reports that known terminal status as informational. A failed condition DIMR job step, MPI preflight, or other child process remains fatal and produces a nonzero final status.

A condition launched with one task is a serial DIMR run and normally reports `MPI: no`. A condition with more than one task should report MPI as enabled and should show the expected rank and OpenMP thread counts.

9 Operational verification

For the first run on a cluster, set `debug = 1` in the `.mm` file. The generated batch, condition, and MorMerge scripts are then retained for inspection.

Check the following values in the job output:

- total allocation nodes;
- derived condition steps per allocation node;
- allocation tasks and CPUs per node;
- nodes, MPI tasks, and CPUs per task for every condition;
- the selected Slurm MPI plugin and loaded MPI module;
- DIMR reporting of MPI and OpenMP status;

- nonzero condition-step exits or failed MPI environment preflights.

If a job remains pending, compare

`slurmConditionsPerNode * slurmTasksPerNode * slurmCpusPerTask`
with the usable CPUs per node and check the site memory request policy. If DIMR does not recognize MPI, verify that `slurmMPI` matches the loaded MPI library and the Slurm plugins installed on the cluster.